

Reviewer Pack — Gowers Norm Inverse Proportional to ACC^0 Circuit Size for Ex...

Entry 331ee94784d2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 22:47:00 UTC

Gowers Norm Inverse Proportional to ACC^0 Circuit Size for Explicit Functions

Entry ID: 331ee94784d2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 22:47:00 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): ACC^0 Circuit Size

Statement:

For a Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the Gowers norm of order 2 ($\|f\|_{U^2}$) is *inversely proportional to the minimal ACC^0 circuit size computing f* . Specifically, $\|f\|_{U^2} \leq 1/\sqrt{S(f)}$ where $S(f)$ is the minimal ACC^0 circuit size.

Rationale (proposer's reasoning):

Gowers norms quantify pseudorandomness, which may correlate with circuit complexity. ACC^0 circuits are limited in their ability to compute functions with high pseudorandomness, creating a potential inverse relationship between these measures.

Taxonomy category: ACC_SIPSER (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3ed0b3545f649ed2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    instances_tested = 30

    def gowers_norm(f):
        norm = 0
        for i in range(2**n):
            sum_val = 0
            for j in range(2**n):
                sum_val += f[i] * f[j] * (-1)**bin(i^j).count('1')
            norm += sum_val**2
        return math.sqrt(norm / (2**(2*n)))

    def dpll_circuit_minimization(f):
        # Simplified DPLL-based heuristic for circuit minimization
        clauses = []
        for i in range(2**n):
            if f[i] != 0:
                clause = [i]
                for j in range(n):
                    if (i >> j) & 1 == 0:
                        clause.append(-j-1)
                clauses.append(clause)
        # Recursive partitioning with clause learning
        def learn_clause(clause):
            return clause

        learned_clauses = []
        while True:
            new_clause = learn_clause(random.choice(clauses))
            if new_clause not in learned_clauses:
                learned_clauses.append(new_clause)
            else:
                break
```

```

    return len(learned_clauses)

def boolean_function():
    return [random.randint(0, 1) for _ in range(2**n)]

metric_value = 0
conjecture_holds = True

for _ in range(instances_tested):
    f = boolean_function()
    norm = gowers_norm(f)
    size = dpll_circuit_minimization(f)
    if norm * size < 1 / math.sqrt(n):
        conjecture_holds = False
        counterexample = "Gowers norm * circuit size < 1/√n"
        break

return {
    "metric_name": "gowers_norm * circuit_size",
    "metric_value": norm * size,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = primes[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with stricter time limits and increased seed diversity to resolve ambiguity

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111065
2	propose	ollama_remote	qwen3:8b	0	0	78247
3	preregistration	ollama_remote	qwen3:8b	0	0	24085
4	novelty	ollama_remote	qwen3:8b	0	0	20578
5	novelty	ollama_remote	qwen3:8b	0	0	14842
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39085
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10250
8	judge	ollama_remote	qwen3:8b	0	0	44881

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 343034 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/331ee94784d2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/331ee94784d2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/331ee94784d2.tar.gz` (if generated)